

# Java

## Лекція 1

# Зміст

- Вступ
- Класи та об'єкти
- Створення об'єктів
- Пакети
- Наслідування
- Абстрактні класи
- Модифікатори доступу

# Вступ

- Java — об'єктно-орієнтована мова програмування.
- Створена у 1995 р. компанією Sun Microsystems, тепер належить Oracle.
- Використання: веб, мобільні, серверні, десктоп.
- Принцип: **“Write once – run anywhere”**.

# Чому java?

- 🌐 Портативність – програми працюють на будь-якій платформі з JVM.
- 🏗️ Об'єктно-орієнтований підхід – зручна модель для великих проектів.
- 📖 Велика стандартна бібліотека – готові класи для роботи з файлами, мережею, колекціями тощо.
- 🔧 Масштабованість та надійність – використовується у великих корпоративних системах (банки, e-commerce).
- 👥 Популярність і спільнота – мільйони розробників та тисячі бібліотек.
- 📱 Універсальність застосування – Android, веб-сервіси, корпоративні додатки, фінансові системи.

# Java платформа

## 1. JDK (Java Development Kit)

Повний набір для розробки

Містить JRE + інструменти розробки

- **javac** - компілятор Java
- **javadoc** - генератор документації
- **debugger** - відладчик
- **jar** - архіватор
- **javap** - дизасемблер

## 2. JRE (Java Runtime Environment)

Середовище для виконання

Містить JVM + стандартні бібліотеки Java

- **Java API** - стандартні бібліотеки (java.lang, java.util, java.io)
- **Deployment технології** - Java Web Start, Java Plug-in
- **UI Toolkits** - AWT, Swing
- **Integration Libraries** - JDBC, JNDI, RMI

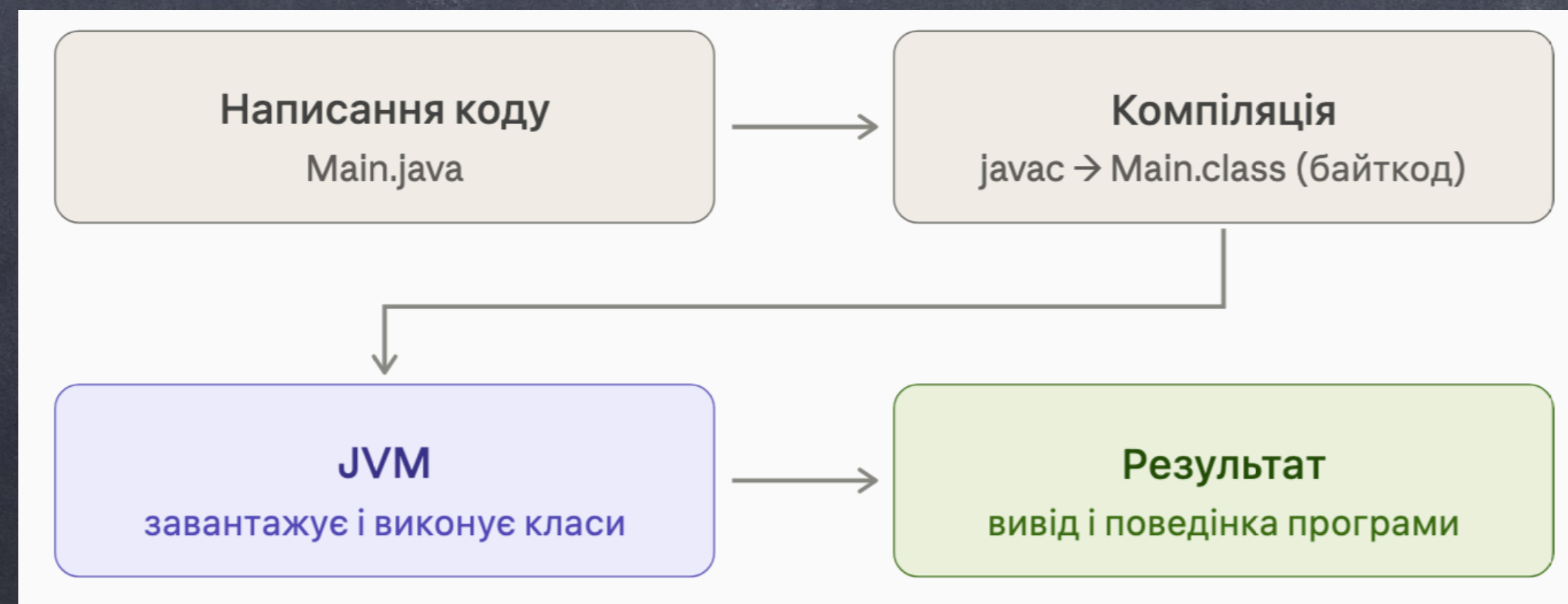
## 3. JVM (Java Virtual Machine)

Власне віртуальна машина

Компілює та інтерпретує байт-код у машинний код

- **Class Loader** - завантаження та верифікація класів
- **Memory Management** - керування пам'яттю (Heap, Stack, Method Area)
- **Execution Engine** - інтерпретатор + JIT компілятор
- **Garbage Collector** - автоматичне прибирання пам'яті

## Процес виконання Java-програми



# Де використовується Java?

## Мобільні застосунки

- **Android**: значна частина застосунків і бібліотек написані на Java.
- **Kotlin** активно розвивається, але Java залишається основою багатьох **Android**-проектів, адже **Kotlin** працює на **JVM** і сумісний із **Java**-кодом.

## Веб-додатки та сервіси

- **Spring Framework** - провідний фреймворк для бекенду.
- Використовується у **Netflix**, **Amazon**, **LinkedIn**, **eBay**.

## Корпоративні системи

- Банківські та фінансові рішення (**JPMorgan**, **Goldman Sachs**).
- **ERP/CRM** для великих компаній.

## Хмарні сервіси

- Java підтримується на **AWS**, **Google Cloud**, **Microsoft Azure**.

## Інші сфери

- Ігри (**Minecraft**)
- Вбудовані системи (**IoT**, смарт-карти).

# Синтаксис java

- Java створювалася як мова без багатьох складних і небезпечних моментів C++, але з легким для розуміння синтаксисом.
- Базові конструкції, знайомі з C++:
- Умовні оператори: `if`, `else`, `switch`
- Цикли: `for`, `while`, `do-while`
- Оператори: арифметичні `+` `-` `*` `/` `%`, логічні `&&` `||` `!`, порівняння `==` `!=` `<` `>` `<=` `>=`
- Масиви: створення і доступ через індекси `int[] arr = new int[10];`
- Методи/функції: визначення і виклик методів з параметрами та поверненням значення
- Класи та об'єкти: `class`, `new`, `this`

# Завдання 0 (повторення синтаксису)



- <https://www.tutorialspoint.com/java/index.htm>
- <https://www.geeksforgeeks.org/java/java/>
- <https://github.com/romanenko-natali/java-2025-syntax>

# Класи та об'єкти

- Клас — шаблон для створення об'єктів.
- Об'єкт — екземпляр класу.

```
class Car {  
    String model;  
    void drive() {  
        System.out.println(model + " is driving...");  
    }  
}
```

# Реалізація принципів ООП

## Інкапсуляція (Encapsulation)

- Приховування внутрішнього стану об'єкта
- Поля робляться `private`, доступ до них — через `public` методи
- Забезпечує контрольований доступ до даних

## Наслідування (Inheritance)

- Створення нових класів на основі існуючих (`extends`)
- Дозволяє повторно використовувати код та розширювати функціональність

## Поліморфізм (Polymorphism)

- Однаковий інтерфейс, різна реалізація
- Використання спільного типу (наприклад, базового класу) для об'єктів різних підкласів
- Ключовий у поєднанні з наслідуванням (перевизначення методів)

## Абстракція (Abstraction)

- Виділення суттєвих характеристик без деталей реалізації
- Досягається через абстрактні класи та інтерфейси
- Дозволяє проектувати систему на рівні концепцій

# Класи

- Поля (*Fields / Attributes*)

- Змінні, що описують стан об'єкта.
- Можуть бути приватними (*private*), захищеними (*protected*), публічними (*public*), доступними в пакеті (без модифікатора доступу).
- Можуть бути статичними (*static*) — спільними для всіх об'єктів класу.

- Методи (*Methods*)

- Визначають поведінку класу.
- Можуть бути статичними (*static*) — викликаються без створення об'єкта.

- Можуть мати модифікатори доступу (*private, protected, public, default*).

- Конструктори (*Constructors*)

- Спеціальні методи для ініціалізації об'єкта при створенні.
- Можуть приймати параметри.
- Можуть бути перевантаженими (один клас — декілька конструкторів з різними параметрами).
- Можуть мати модифікатори доступу (*private, protected, public, default*).

<https://onlcomplier.com/java>

Code1

<https://www.jdoodle.com/online-java-compiler>

```
public class Car {  
    // Поля  
    private String model;  
    public int year;  
    static int totalCars = 0; // статичне поле  
  
    // Конструктор  
    public Car(String model, int year) {  
        this.model = model;  
        this.year = year;  
        totalCars++;  
    }  
  
    // Методи  
    public void drive() { // нестатичний метод  
        System.out.println(model + " is driving.");  
    }  
  
    public static int getTotalCars() { // статичний метод  
        return totalCars;  
    }  
}
```

# Static

- Поле або метод належить класу, а не конкретному об'єкту.
- Не потрібно створювати об'єкт, щоб звернутися до `static` члена.
- Статичні методи не можуть безпосередньо звертатися до нестатичних полів або методів класу. Причина: статичний метод належить класу, а нестатичні поля/методи належать конкретному об'єкту.
- Зручно для констант, утилітних методів і лічильників об'єктів.

# Приклад використання static

## Code2

```
class Counter {
    int instanceVar;    // нестатичне поле
    static int staticVar = 0; // статичне поле спільне для всіх об'єктів

    Counter(int value) {
        this.instanceVar = value;
        staticVar++;    // збільшуємо статичне поле
    }

    static void showStatic() {
        System.out.println("Static count: " + staticVar);

        // System.out.println(instanceVar); // ❌ помилка! Не можна
        // звертатися до нестатичного поля без об'єкта

        // Правильний доступ через об'єкт
        Counter temp = new Counter(0);
        System.out.println("Access instanceVar via object: " +
temp.instanceVar);
    }
}
```

```
void showInstance() {
    System.out.println("Instance value: " + instanceVar);
    System.out.println("Static count from instance: " + staticVar);
}

public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter(10);
        Counter c2 = new Counter(20);

        c1.showInstance();
        c2.showInstance();

        // Виклик статичного методу без об'єкта
        Counter.showStatic();
    }
}
```

# Конструктор

Спеціальний метод для ініціалізації об'єкта.

Має таку ж назву, як і клас.

```
class Car {  
    String model;  
    Car(String m) {  
        model = m;  
    }  
}
```

# Створення об'єктів (1)

## ◆ 1. Створення через конструктор

```
public class Car {  
    String model;  
  
    public Car(String model) {  
        this.model = model;  
    }  
}
```

```
public class Main {  
    public static void main(String[]  
args) {  
        Car car1 = new Car("Toyota");  
        System.out.println(car1.model);  
    }  
}
```

# Створення об'єктів (2)

## ◆ 2. Створення через статичний фабричний метод

Використовується, коли вибір або підготовка об'єкта — теж частина задачі, а не тільки ініціалізація. Конструктор при цьому часто роблять приватним, щоб фабрика лишалась єдиним входом. Переваги: контроль створення об'єктів, можливість кешування, обмеження кількості екземплярів, застосовується у патернах проектування (Singleton, Factory, ...). Фабрик у класу може бути кілька, і їхні імена документують різні сценарії створення (наприклад, `createNew(...)` — з перевітками для нових даних, `load(...)` — для вже збережених) — перевантажені конструктори так не вміють, бо не мають імен. Є ще `Builder` — коли об'єкт збирається покроково через ланцюжок методів: `HttpRequest.newBuilder().uri(...).build()`.

```
public class Car {  
    private String model;  
  
    // Приватний конструктор  
    private Car(String model) {  
        this.model = model;  
    }  
  
    // Статичний метод для створення об'єкта  
    public static Car createCar(String model) {
```

```
        return new Car(model);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car car2 = Car.createCar("BMW");  
        System.out.println(car2.model);  
    }  
}
```

# Package, imports

- `package` — групування класів (аналог папки).
- `import` — дозволяє підключати класи.

```
package vehicles;
```

```
public class Car { }
```

```
import vehicles.Car;
```

# Source root

- Папка, з якої починається структура пакетів

```
/Users/nataliia/IdeaProjects/java-2025-syntax ← project root
└─ src/main/java                               ← source root
    └─ ua/
        └─ university/
            └─ Main.java
            └─ Main.class
```

## ◆ 1. Компіляція

- Можна перебувати будь-де, головне вказати шлях до `.java` файлу:

```
cd
/Users/nataliia/IdeaProjects/java-2025-syntax
javac
src/main/java/ua/university/Main.java
```

- `.class` файл з'явиться у тій же папці (ієрархія пакетів збережеться):

```
src/main/java/ua/university/Main.class
```

## ◆ 2. Запуск

### Варіант А: перебуваємо у source root

```
cd
src/main/java
java
ua.university.Main
```

✓ JVM бачить `ua/university/Main.class` Відносно source root.

### Варіант В: перебуваємо у project root

```
cd
/Users/nataliia/IdeaProjects/java-2025-syntax
java
-cp src/main/java ua.university.Main
```

✓ JVM шукає клас у папці, зазначеній через `-cp` (classpath).

# Наслідування

```
class Product {
    protected String name;
    protected double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public void showInfo() {
        System.out.println(name + " коштує " + price + " грн");
    }
}

class Electronics extends Product {
    private int warrantyMonths;

    Electronics(String name, double price, int warrantyMonths) {
        super(name, price);
        this.warrantyMonths = warrantyMonths;
    }

    @Override
    public void showInfo() {
```

```
        System.out.println(name + " (електроніка) коштує " + price
+
        " грн, гарантія: " + warrantyMonths + " місяців");
    }
}
```

```
class Food extends Product {
    private String expirationDate;

    Food(String name, double price, String expirationDate) {
        super(name, price);
        this.expirationDate = expirationDate;
    }

    @Override
    public void showInfo() {
        System.out.println(name + " (їжа) коштує " + price +
        " грн, придатна до: " + expirationDate);
    }
}
```

# Анотація `@Override` — страховка від тиші

- Нічого не змінює в роботі програми
- Це контракт з компілятором: «цей метод перевизначає метод предка — перевір»
- Одрук у назві (`showInfo`), інші параметри, змінена сигнатура предка → без анотації тихо з'являється НОВИЙ метод; з анотацією — помилка компіляції
- Правило: пишемо `@Override` над кожним перевизначенням. **Завжди.**

```
class Electronics extends Product {  
  
    @Override  
    void showInfo() { ... }  
    // компілятор ПЕРЕВІРИТЬ, що ми  
    // справді перевизначаємо  
}
```

Питання: що станеться, якщо метод предка зробити `private`, а `@Override` у нащадка прибрати?

# Модифікатори доступу

| Модифікатор | Клас | Пакет | Підклас | Весь код |
|-------------|------|-------|---------|----------|
| private     | ✓    | ✗     | ✗       | ✗        |
| default     | ✓    | ✓     | ✗       | ✗        |
| protected   | ✓    | ✓     | ✓       | ✗        |
| public      | ✓    | ✓     | ✓       | ✓        |

```
package animals;

public class Animal {
    private String privateName = "Private Animal";
    String defaultName = "Default Animal";
    protected String protectedName = "Protected Animal";
    public String publicName = "Public Animal";

    private void privateMethod() {
        System.out.println("Private method");
    }

    void defaultMethod() {
        System.out.println("Default method");
    }

    protected void protectedMethod() {
        System.out.println("Protected method");
    }

    public void publicMethod() {
        System.out.println("Public method");
    }

    public void showNames() {
        System.out.println("Private: " + privateName);
        System.out.println("Default: " + defaultName);
        System.out.println("Protected: " + protectedName);
        System.out.println("Public: " + publicName);
    }
}
```

```
package zoo;

import animals.Animal;

public class ZooKeeper extends Animal {
    public void showAccess() {
        // defaultName is NOT accessible in another package
        // privateName is not accessible
        // protectedName is accessible because we are a subclass
        System.out.println(protectedName);
        System.out.println(publicName);

        // defaultMethod();    // not accessible
        protectedMethod();
        publicMethod();
    }

    public void tryAccessAnimal() {
        Animal a = new Animal();
        // a.protectedName; // NOT accessible via object reference from another package
        // a.publicName;    // accessible
        System.out.println(a.publicName);
    }
}
```

```
package animals;

public class Dog extends Animal {
    public void showAccess() {
        // privateName is not accessible
        // defaultName is accessible because we are in the same package
        System.out.println(defaultName);
        System.out.println(protectedName);
        System.out.println(publicName);

        // privateMethod();    // not accessible
        defaultMethod();
        protectedMethod();
        publicMethod();
    }
}
```

```
package ua.university;

import animals.Animal;
import animals.Dog;
import zoo.ZooKeeper;

/*
 * nataliia
 */
public class Main {
    /*
     * nataliia
     */
    public static void main(String[] args) {

        System.out.println("Project works");

        Animal a = new Animal();
        Dog d = new Dog();
        ZooKeeper z = new ZooKeeper();

        System.out.println("Animal access:");
        a.showNames();

        System.out.println("\nDog access:");
        d.showAccess();

        System.out.println("\nZooKeeper access:");
        z.showAccess();
        z.tryAccessAnimal();
    }
}
```

# Практичні рекомендації (доступ)

## 1 Поля / змінні

- Завжди **private** за замовчуванням.
- Виняток:
  - константи — `public static final`
    - `public static final int MAX_SIZE = 100;`
- Мета: інкапсуляція → контроль доступу до внутрішнього стану об'єкта.

# Практичні рекомендації (доступ)

## 2 Геттери та сеттери

Робимо **public** лише якщо дійсно потрібно.

```
private int age;
```

```
public int getAge() { return age; }
```

```
public void setAge(int age) { this.age = age; }
```

Уникаємо непотрібних сеттерів, щоб об'єкти залишалися контрольованими або незмінними.

# Практичні рекомендації (доступ)

## 3 Default / package-private

Використовуємо без модифікатора для членів/класів, які повинні бути видимі тільки в пакеті.

Мета:

Інкапсуляція пакету → приховуємо внутрішню реалізацію від зовнішнього світу.

Добре для допоміжних класів, внутрішньої логіки чи утиліт, не призначених для публічного використання.

# Практичні рекомендації (доступ)

## 4 `protected`

Використовуємо для класів, що планується наслідувати.

Уникаємо `protected` для полів, якщо підклас не потребує прямого доступу.

Якщо нащадку потрібен доступ — краще `protected` метод, а поле лишити `private`.

Наслідування використовуємо, коли підклас справді є спеціалізацією батька; якщо потрібен лише чужий код — краще композиція.

## 5 `public`

Використовуємо тільки для:

API, яке повинно бути доступне скрізь.

Методів, які призначені для використання поза пакетом.

Поля не робимо `public`; краще через геттери/сеттери.

# Практичні рекомендації (доступ)

## 6 Загальні рекомендації

Мінімізуємо доступ → починаємо з **private**.

Відкриваємо доступ тільки коли необхідно.

Інкапсуляція допомагає з підтримкою, рефакторингом і відлагодженням.

Запитання, яке варто ставити: "Хто дійсно потребує доступ до цього поля, методу, класу?"

# Абстрактний клас (екземпляр НЕ МОЖНА СТВОРИТИ)

- `abstract` = «екземплярів не буває, тільки нащадки»
- Використовуємо, коли базовий клас — спільний код, а не самотійна сутність («продукт взагалі» не існує — існує електроніка, їжа...)

```
public abstract class Product {  
    protected String name;  
    protected double price;  
  
    protected Product(String name, double price) { ... }  
}  
  
Product p = new Product("?", 0);    // ПОМИЛКА КОМПІЛЯЦІЇ  
Product p = new Electronics(...);  // ок: створюємо нащадка
```

# Абстрактні методи

- Методи без реалізації.  
Реалізація буде в нащадках:

```
public abstract double  
calculateFinalPrice();
```

- Якщо хоча б один метод в класі абстрактний, то клас повинен бути оголошений теж як **abstract**

```
public abstract class Product {  
    protected String name;  
    protected double price;  
  
    protected Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public abstract double calculateFinalPrice();  
}  
  
public class Electronics extends Product {  
    public Electronics(String name, double price){ super(name, price); }  
  
    @Override  
    public double calculateFinalPrice() { return price * 1.10; }  
}  
  
public class Food extends Product {  
    public Food(String name, double price) { super(name, price); }  
  
    @Override  
    public double calculateFinalPrice() { return price * 0.80; }  
}
```

# Підсумок

- Java – мова C-подібна, але зі спрощеним і безпечнішим синтаксисом.
- Ми розглянули створення класів та використання конструкторів для ініціалізації об'єктів.
- Pozнайomились зі статичними методами, які можна викликати без створення об'єкта.
- Вивчили наслідування, яке дозволяє створювати підкласи і повторно використовувати код батьківського класу.
- Розібралися з модифікаторами доступу (*private, default, protected, public*) і їх впливом на видимість полів та методів.
- З'ясували, що абстрактний клас не обов'язково буде мати абстрактний метод.
- Підкреслили принцип інкапсуляції: обмежуємо доступ до внутрішнього стану об'єкта і забезпечуємо контрольований доступ через методи.